



System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

First-Come,First-Serve CPU Scheduling Algorithm

Completed

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i;
5
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8
9     int AT[n], BT[n], CT[n], TAT[n], WT[n];
10
11
12     for(i = 0; i < n; i++) {
13         printf("Enter Arrival Time and Burst Time for P");
14         scanf("%d %d", &AT[i], &BT[i]);
15     }
16
17     CT[0] = AT[0] + BT[0];
18
19     for(i = 1; i < n; i++) {
20         if(CT[i - 1] < AT[i]) {
21             CT[i] = AT[i] + BT[i];
22         } else {
23
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

```
3
0 2
1 2
2 3
```

Expected Output :

```
Enter number of
processes:
Enter Arrival Time and
Burst Time for P1:
Enter Arrival Time and
Burst Time for P2:
Enter Arrival Time and
```



System -
Coding |
REC | 3
Feb

Course Progress

Go Back

Back To Practice

Save

Q1

Select Language : C (GCC 9.2.0)

Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)
- Waiting Time (WT) =

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, time = 0, completed = 0;
5
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8
9     int AT[n], BT[n], CT[n], TAT[n], WT[n];
10    int visited[n];
11
12    for(i = 0; i < n; i++) {
13        printf("Enter Arrival Time and Burst Time for P");
14        scanf("%d %d", &AT[i], &BT[i]);
15        visited[i] = 0;
16    }
17
18    while(completed < n) {
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

Expected Output :

4

0 6

0 8

0 7

0 3

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Output :

1 of 1

01-05-2026, 10:54 am

**FACE Prep**

System -
Coding |
REC | 3
Feb

Course Progress

Go Back

Back To Practice

Save

Q1

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, time = 0, completed = 0;
5
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8
9     int AT[n], BT[n], P[n], CT[n], TAT[n], WT[n], visited[n];
10
11     for(i = 0; i < n; i++) {
12         printf("Enter AT, BT and Priority for P%d: \n", i+1);
13         scanf("%d %d %d", &AT[i], &BT[i], &P[i]);
14         visited[i] = 0;
15     }
16
17     while(completed < n) {
18         int idx = -1;
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

3
0 5 2
1 3 1
2 2 3

Expected Output :

Enter number of
processes:
Enter AT, BT and Priority
for P1:
Enter AT, BT and Priority
for P2:
Enter AT, BT and Priority

Output :

Enter number of processes:

Non-Preemptive Priority Scheduling algorithm

Completed

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value